SmartPOS project report

How it works, what is left, and how to make it more secure

Where the project stands

SmartPOS already covers the core of running a shop: selling, taking payment, printing receipts, counting stock, running a kitchen screen, showing customers their order and managing staff. What separates it from a product you can sell is mostly finishing work: one workflow bug, some screens that still show sample data, tax, and a set of security protections that need adding before real shops rely on it.

Priority	What to do	Why it matters
1	Fix orders that are paid before they are cooked	They skip the kitchen, so no ready alert goes out.
2	Add the missing security basics (section 4)	The API has no general rate limit, and any website may call it.
3	Replace sample data on Finance, Winner and Services	An owner could mistake sample figures for real money.
4	Build tax properly	Receipts and checkout still guess 18% VAT.
5	Add tests for money and stock	A mistake here costs shops real money without anyone noticing.

> **Before the next test:** commit the app and backend work, then run **npm run migration:run** in pos-backend and restart it. The TIN, product details, item note and Served changes all need their database migrations.

How this report was made

It comes from reading the app and backend code on 1 November 2026. The app was not run, so a screen listed as built has been written and type-checked, not necessarily tested on a device. The newest work (kitchen notes, the ready alert, stock screens) is the least tested.

1. How the project works

There are two programs. The **app** (smartPOS, built with Expo) runs on phones, tablets, the web and desktop. The **backend** (pos-backend, built with NestJS) stores everything in a database and checks every action. Devices ask the backend to do things over the internet, and it pushes live updates back over a socket connection.

The parts of the system

Screen	Who uses it	What it does
Till	Cashiers and waiters	Sells products, takes notes for the kitchen, takes payment, prints receipts.
Kitchen display	Cooks	Shows today's food orders: New, Preparing, Ready, Served. Older orders are in History.
Customer display	Customers (nobody signs in)	Shows the order, total, MoMo code and a thank-you, live from the paired till.
Owner area	Owners and trusted staff	Products, stock, staff, roles, devices, finance and company settings.
Backend and database	Runs in the background	Stores the data, checks permissions and sends live updates.

Setting up a device

Device asks to join → Owner sees a live request → Owner approves it and picks its type → Device works as a till, kitchen or customer display

Making a sale

Worker signs in with a PIN → Adds products by tap or barcode → Customer sees items live → Kitchen items ask for a note

Cart warns if stock is short → Checkout creates the order → Pays by cash, MoMo, card or credit → Receipt, thank-you, stock drops

Stock goes down when the order is paid, not at checkout, so an abandoned order never touches stock. If a sale reaches a product's alert level, owners get a live low-stock notice.

Food in the kitchen

Only Show in kitchen items are sent → New: order appears with photo and notes → Preparing: cook taps Start preparing → Ready: cook taps Mark ready

Alert and sound reach the waiter → Nobody online? Everyone active is alerted → Someone taps Mark served on a till → Sound stops on all devices

Running the business

Area	What the owner can do
Products	Add and edit products with photos, prices, unit, stock, expiry, supplier and kitchen setting. Deactivate them instead of deleting, so old receipts stay correct.
Stock	See stock levels, restock deliveries, view low stock and expiring items, and read each product's stock history.
Accountability	See who added or changed a product and exactly what changed, in Activity Logs.
Staff	Add employees, build roles from switches, and give each worker exactly what they need.
Money	Verify each worker's takings and track customers who owe money.
Company	Keep the name, address, phone, logo and TIN that appear on receipts.

2. What you are left with

Fix these first

Problem	What happens today	What to do
Paid orders skip the kitchen	A paid order is final, so one paid at checkout goes straight to Served. It cannot be marked Ready and no alert goes out.	Keep the kitchen's progress separate from payment, with its own status on each order.
Sample data on real screens	Finance, Winner, the Services pages and the owner layout import sample data. Finance loads no real data at all.	Connect them to real data, or hide each page until it is ready.
Tax	Orders are saved with tax 0, and the receipt, cart and checkout work out 18% themselves. You chose to build this later.	Save a tax type per product and the tax amount per sold item, then remove the guess.
Unfinished orders at midnight	Orders still cooking leave the board at 00:00 and cannot be moved from History.	Keep unfinished orders on the board until they are done.
Hold sale button	The Hold button in the cart does nothing when tapped.	Build it, or remove it.

Finish and clean up

Item	Detail
One order number	The same order appears in several formats on the receipt, kitchen and order lists. A short shop number like #1025 would be clearest.
Fake content left	"John Doe" in the Services bar, a stranger's photo in profile2, and "Order #12345" on the tables receipt.
Selling part of a unit	Stock allows decimals, but the till sells whole numbers only, so 1.5 kg cannot be sold yet.
One note per product line	Two burgers share one kitchen note. Separate lines would need a change to the cart.
Line ids on the customer display	Rows are matched by product name, so two lines with the same name would clash.
No-popup rule	About 17 confirm or alert popups remain, against your rule of one tap and a Toast.
Brand colours	Many off-brand colours remain, mostly in Finance, Services, the tables receipt and the orders list.
Tests	The app has no automated tests. The backend has 25 test files, not reviewed here. Start with totals, payments and stock.
Tidy code	Remove debug logs and split the largest files, which run to about 900 to 1,250 lines.

3. Unique features to build

Most point-of-sale apps treat a shop as a list of items with a price. These ideas would make SmartPOS feel built for how shops in Rwanda really work. The order is a suggestion, starting with what brings the most value for the least work.

Kitchen display

Feature	What it gives you
New-order sound and flash	Cooks notice an order without watching the screen.
Clear cancelled message	Today a cancelled order just disappears, so food could still be cooked.
Connection-lost banner	A disconnected screen looks normal while orders stop arriving.
Table number or customer name, dine-in or takeaway	Cooks and waiters know who each order is for.
Tick off single items	Shows progress on large orders.
Stations, such as grill and bar	Each screen shows only what that station makes.
Prep-time reports	Shows average cooking time and the slowest dishes.
Large-text and dark modes, spoken alerts, printed tickets	Suits wall screens, dim kitchens and noisy rooms.

The whole product

Feature	What it gives you
Local selling units	Already started: crate, heap, jerrycan, tray and tot. Add reports per unit, and a saved list of custom units so reports stay clean.
Sell a crate, track the bottles	Convert between units, so a crate bought becomes 24 bottles sold. Bars and shops would value this highly.
MoMo payment confirmation	Confirm a MoMo payment automatically instead of the cashier checking a phone.
Works without internet	Keep selling through an outage and sync afterwards. This is a real advantage where connections drop.
Credit book with reminders	Track who owes what, and send SMS or WhatsApp reminders.
Kinyarwanda language	Lets any member of staff use the app comfortably.
Daily summary to the owner	Sales, cash, low stock and the best worker by SMS or WhatsApp each evening.
Smart reorder suggestions	Suggest what to buy from sales speed and expiry dates.
Theft and mistake alerts	Flag repeated cancellations, stock counts that do not match, and cash shortfalls by worker.
Customer loyalty	Points or a free item after a number of visits, linked to a phone number.
Several branches	One owner view across shops. Decide this early, because stock would need to be kept per branch.

4. Security

Already in place

Protection	Detail
Passwords and PINs	Stored only as bcrypt hashes, and never loaded unless needed.
Login attempts	Failed email and PIN logins are limited, per email or device.
Live permission checks	Roles and active status are read on each request, so removing access works at once.
Data separated by company	Queries are limited to the signed-in company.
Device approval	A device must be approved by the owner and has its own secret.
Input checks	The API rejects fields it does not expect, and limits uploads by type and size.
Change history	Who created, edited or deleted a record is logged.
Login token on devices	Phones and tablets now keep it in secure storage.
Secrets	The .env files are not tracked in git in either project.

What to add

Priority	Add this	Why
High	Rate limit the whole API (@nestjs/throttler)	Only login is limited today, so other routes can be flooded.
High	Restrict CORS to your own web addresses	CORS is open, so any website's scripts may call the API from a browser.
High	Add helmet for security headers	A quick, standard protection that is not installed yet.
High	Hide the API docs in production	/doc is set up without any environment check, which gives attackers a full map of the API.
High	HTTPS only, with no ngrok in production	Logins and money data must always be encrypted in transit.
High	Daily database backups, with a tested restore	Sales records must survive a server failure, not just an attack.
High	Rotate secrets before launch, with a long JWT secret	Anything used during development should be treated as exposed.
Medium	Shared login limits, for example in Redis	The login limit lives in one process's memory, so it resets on restart and is not shared between servers.
Medium	Short-lived tokens with refresh tokens	Only access tokens exist; the refresh code is commented out.
Medium	Secure cookies on web and desktop	Tokens there sit in browser storage, which a script could read.
Medium	Two-step login for owners	An owner's account controls staff, money and devices.
Medium	Auto-lock on tills and kitchen screens	Shared screens stay signed in when staff walk away.
Medium	Log money actions and failed logins	Keep a record of cancellations and refunds, and watch for repeated failures.
Medium	Check dependencies with npm audit	Known flaws in packages are a common way in.
Normal	Error monitoring and server alerts	You find out about problems before shops call you.
Normal	Limit and protect customer data	Credit records hold names and phone numbers. Check Rwanda's data protection law.
Normal	Independent security test before launch	Someone outside the project will find what you have missed.

Check with a professional: before selling to tax-registered businesses, confirm the Rwanda Revenue Authority's rules for electronic receipts (EBM). This report only flags the question, and it may shape how you build tax.